

1. A BST keeps all left values smaller and right values larger, so an in-order traversal prints everything in sorted order. A min-heap only makes sure each parent is smaller than its children, but not between subtrees, so you can't print keys from a min-heap in sorted order in $O(n)$, it would take $O(n \log n)$.

2. When inserting a new value, we compare it to nodes until we find where it fits, If we check k nodes while inserting, searching that same value later checks $k + 1$ nodes (because now the node itself exists).

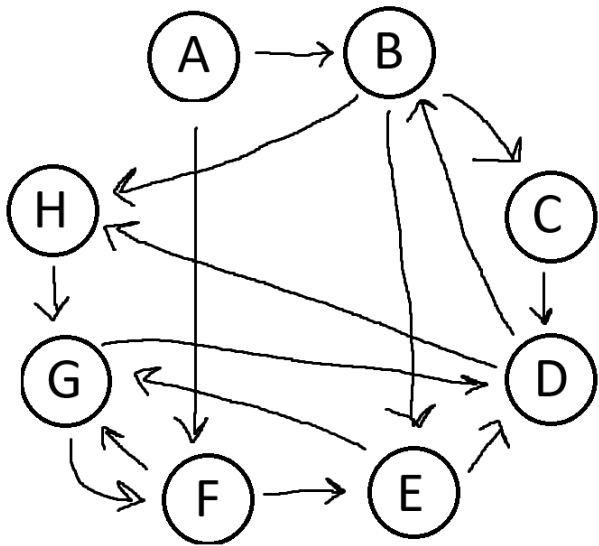
3. If a node has two children, its successor (next largest) is the smallest in the right subtree, which means it has no left child. Its next smallest is the largest in the left subtree, so it has no right child

4. Each user is a node and friendships are edges. Use BFS starting from one user, visit all their friends, then friends of friends, ect. The first time you reach the target the user gives the shortest connection path

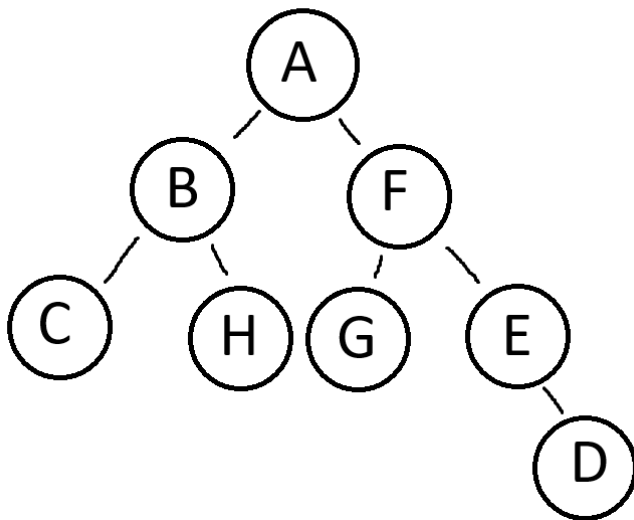
5. Use BFS if you want the shortest path to a file. Use DFS if you just want to explore everything or have limited memory. Most file searches use DFS since it's simple and doesn't use as much memory

6.

a)



b)

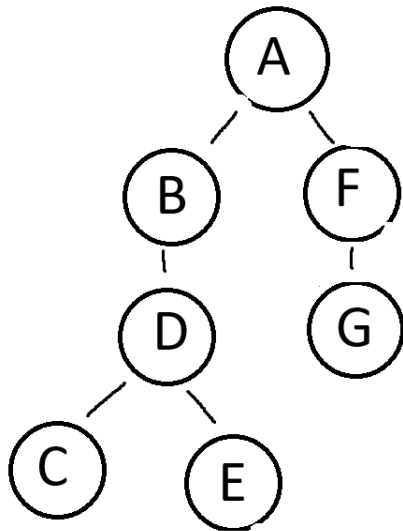


7. Use BFS because it finds the shortest path from start to finish. DFS might reach the goal but not with the shortest route. BFS is better for shortest path, DFS is better for saving memory

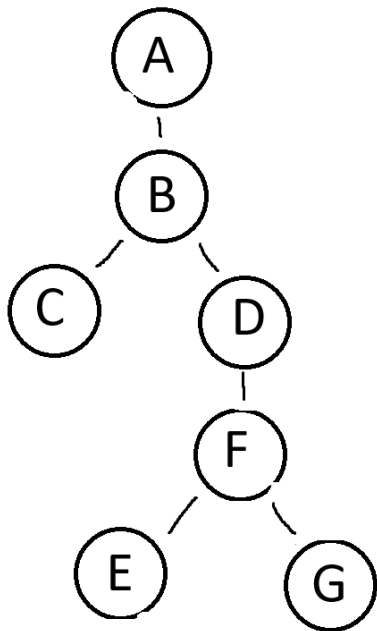
8.

a)

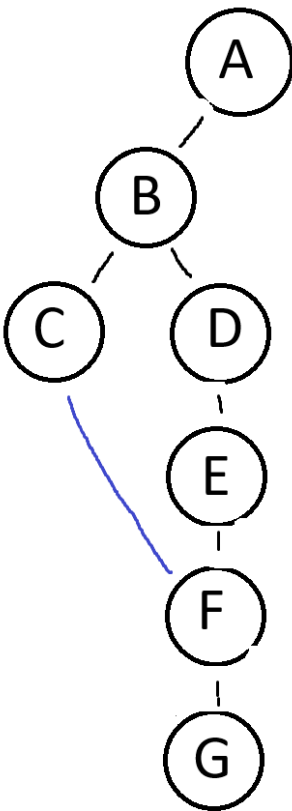
BFS:



DFS:



b)



There is a cycle at fc, highlighted with the blue line.